

文章笔记

Harness Design for Long-Running Application Development

基于公开课程资料整理

2026 年 3 月 24 日

文章作者: Prithvi Rajasekaran (Anthropic Labs)

发布日期: 2026 年 3 月 24 日

阅读时长: 阅读时长: 约 25 分钟

文 章 链 接: <https://www.anthropic.com/engineering/harness-design-long-running-apps>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言	2
2 朴素实现为何失败	2
2.1 上下文窗口管理问题	2
2.2 自我评估偏差	2
2.3 本章小结	3
3 前端设计：让主观质量可评分	3
3.1 四维评分标准	3
3.2 Generator-Evaluator 闭环	4
3.3 迭代过程中的观察	4
3.4 本章小结	4
4 扩展到全栈编码	5
4.1 三 Agent 架构	5
4.2 Sprint 合同机制	5
4.3 Evaluator 的调优挑战	5
4.4 Retro Game Maker 对比实验	6
4.5 本章小结	6
5 迭代简化 Harness	7
5.1 Opus 4.6 带来的变化	7
5.2 移除 Sprint 结构	7
5.3 DAW 构建实验	8
5.4 本章小结	8
6 总结与延伸	9
6.1 作者的核心洞察	9
6.2 结构化提炼	9
6.3 拓展阅读	9

1 引言

本文来自 Anthropic Labs 的工程师 Prithvi Rajasekaran，探讨了如何通过 **multi-agent harness** 设计来突破 Claude 在两类任务上的性能天花板：**高质量前端设计**（主观审美判断）和**长时间自主软件工程**（可验证的正确性与可用性）。作者从 GAN（生成对抗网络）中汲取灵感，设计了 generator-evaluator 分离架构，并最终扩展为 planner-generator-evaluator 三 agent 流水线，在多小时的自主编码会话中产出完整的全栈应用。

核心论点

“Every component in a harness encodes an assumption about what the model can’t do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve.”

Harness 中的每个组件都编码了一个关于模型能力边界的假设。随着模型迭代，这些假设需要持续验证和更新。

2 朴素实现为何失败

在进入 harness 设计之前，作者首先剖析了“单 agent 直接执行”模式的两个系统性失败模式。

2.1 上下文窗口管理问题

模型在执行长任务时，随着 context window 逐渐填满，输出的连贯性会显著下降。更微妙的是，Claude 会表现出一种被称为 **context anxiety** 的行为：当模型“感知到”自己接近上下文长度限制时，会提前草草收尾。

Context Reset vs. Compaction

作者对比了两种应对上下文膨胀的策略：

Compaction (压缩)：对对话历史进行摘要，缩短后继续。保留了连续性，但无法解决 context anxiety——因为 agent 仍然“记得”自己已经工作了很久。

Context Reset (重置)：彻底清空上下文，启动全新 agent，配合结构化 handoff artifact 传递状态。提供了一个“干净的开始”，但增加了编排复杂度和 token 开销。

在早期测试中，Claude Sonnet 4.5 的 context anxiety 严重到 compaction 不足以应对，因此 context reset 成为 harness 设计的必要组件。

2.2 自我评估偏差

Agent 的自我评估陷阱

“When asked to evaluate work they’ve produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre.”

当要求 agent 评估自己产出的工作时，它们倾向于自信地给出正面评价——即使在人类观察者看来质量明显平庸。这个问题在主观任务（如设计）上尤为突出，因为不存在像软件测试那样的二值化验证手段。

关键洞察在于：**将生成和评估分离**后，调优一个持怀疑态度的 evaluator 远比让 generator 对自身工作保持批判性更容易。一旦外部反馈存在，generator 就有了具体的迭代目标。

2.3 本章小结

朴素的单 agent 实现面临两个结构性困境：(1) 上下文窗口膨胀导致的连贯性退化和 context anxiety；(2) 自我评估偏差导致的质量停滞。这两个问题共同构成了引入 multi-agent harness 的动机。

3 前端设计：让主观质量可评分

作者从前端设计任务入手，因为自我评估问题在这里最为突出。在没有任何干预的情况下，Claude 倾向于产出安全、可预测但视觉上平淡无奇的布局。

GAN 的启发

生成对抗网络（GAN）的核心思想是让 generator 和 discriminator 在对抗中共同进步。作者将这一思想迁移到 LLM agent 系统中：generator 负责生成前端页面，evaluator 负责评判质量并给出具体反馈，形成迭代改进的闭环。

3.1 四维评分标准

两个关键洞察驱动了 harness 的设计：(1) 虽然审美不能完全量化，但可以通过编码设计原则来使其“可评分”；(2) 将生成和评分分离可以创造驱动 generator 提升的反馈闭环。

作者定义了四个评分维度，同时写入 generator 和 evaluator 的 prompt 中：

维度	定义	权重
Design Quality	设计是否感觉像一个连贯的整体？颜色、排版、布局、图像是否创造了独特的情绪和身份感？	高
Originality	是否有定制化的设计决策？还是模板布局、库默认值和 AI 生成模式的堆砌？	高
Craft	技术执行：排版层次、间距一致性、色彩和谐、对比度。这是能力检查而非创意检查。	低
Functionality	独立于审美的可用性。用户能否理解界面功能、找到主要操作、完成任务？	低

表 1: 四维评分标准及其权重分配

权重设计的理由

作者有意加大 Design Quality 和 Originality 的权重，降低 Craft 和 Functionality 的权重。原因是 Claude 在技术执行（Craft）和功能实现（Functionality）方面本身表现不错，但在设计感和原创性方面常常产出“高度通用的 AI slop”。评分标准明确惩罚这些通用模式（如“紫色渐变覆盖白色卡片”），并通过更高权重推动模型承担更多审美风险。

3.2 Generator-Evaluator 闭环

整个闭环基于 Claude Agent SDK 构建：

1. **Generator** 根据用户 prompt 生成 HTML/CSS/JS 前端页面
2. **Evaluator** 通过 **Playwright MCP** 与活页面交互——自主导航、截屏、仔细研究实现——然后为每个维度打分并撰写详细评论
3. 评论反馈回 generator 作为下一轮迭代的输入
4. Generator 做出策略决策：如果分数趋势良好，**refinement**（在当前方向上优化）；如果方向不对，**pivot**（转向完全不同的审美）
5. 重复 5–15 轮迭代

每轮完整运行耗时较长（evaluator 需要实际操作页面），整体可达 **4 小时**。

Prompt 措辞的意外影响

评分标准中的用语会以未预期的方式引导 generator 的输出。例如，包含 “the best designs are museum quality” 这样的短语会推动设计向特定的视觉风格收敛。这提示我们 prompt 中的隐喻和形容词本身就是强信号。

3.3 迭代过程中的观察

- 分数通常随迭代提升，但**不总是线性的**——作者经常发现中间轮次的产出优于最终轮次
- 实现复杂度倾向于逐轮增加，generator 在 evaluator 反馈下追求更 ambitious 的方案
- 即使在第一轮（无 evaluator 反馈），有评分标准的产出已显著优于无任何 prompt 工程的 baseline
- Evaluator 的校准使用了 **few-shot examples**（包含详细分数拆解），以确保判断与作者偏好对齐，并减少跨迭代的分数漂移

荷兰艺术博物馆的创意跃迁

作者用 “为一家荷兰艺术博物馆创建网站” 作为 prompt。前 9 轮产出了一个干净的深色主题着陆页——视觉上精致但在预期之内。到第 10 轮，generator 彻底放弃了当前方向，将网站重新构想为一个**空间体验**：用 CSS perspective 渲染的棋盘格地板 3D 房间，艺术品以自由位置悬挂在墙上，通过门廊导航而非滚动或点击切换画廊。这种创意跃迁是作者此前从未在单次生成中看到的。

3.4 本章小结

通过将主观审美判断分解为四个可评分维度，并建立 generator-evaluator 对抗闭环，作者成功将 Claude 的前端设计质量从 “技术上可用但视觉平淡” 提升到了具有审美风险和创意跃迁的水平。关键在于：（1）评分标准本身就是强 prompt 信号；（2）分离生成与评估使得调优 evaluator 的 “怀疑态度” 远比调优 generator 的 “自我批判” 更可行。

4 扩展到全栈编码

将 GAN 启发的 generator-evaluator 模式从前端设计扩展到全栈开发是自然的——代码审查和 QA 在软件开发生命周期中扮演着与 design evaluator 完全相同的结构性角色。

4.1 三 Agent 架构

Planner – Generator – Evaluator 流水线

作者设计了三个 agent persona，每个 agent 针对先前观察到的特定失败模式：

Planner：将用户的 1-4 句简短 prompt 扩展为完整的产品规格。被指示保持高层次的产品上下文和技术设计，而非详细的技术实现——因为如果 planner 在技术细节上犯错，错误会级联到下游实现。同时被要求在规格中融入 AI 功能。

Generator：以 sprint 为单位工作，每次从规格中取一个 feature 实现。技术栈：React + Vite + FastAPI + SQLite/PostgreSQL。每个 sprint 结束时进行自我评估，然后交给 QA。拥有 git 进行版本控制。

Evaluator：通过 Playwright MCP 像真实用户一样点击运行中的应用，测试 UI 功能、API 端点和数据库状态。按维度打分，任一维度低于阈值则 sprint 失败，generator 收到详细反馈。

4.2 Sprint 合同机制

从高层规格到可测试实现的桥梁

产品规格是有意保持高层次的（避免 planner 在细节上犯错导致级联失败）。为了弥合用户故事与可测试实现之间的间隙，作者引入了 **sprint contract** 机制：

1. Generator 提出本 sprint 要构建的内容及成功验证标准
2. Evaluator 审查提案，确保 generator 构建的是正确的东西
3. 双方迭代直到达成一致
4. Generator 按合同实现
5. Evaluator 按合同验收

Agent 之间的通信完全通过文件进行：一个 agent 写文件，另一个读文件并通过修改或新建文件来回应。

4.3 Evaluator 的调优挑战

Claude 天生不是好 QA

开箱即用的 Claude 是一个很差的 QA agent。在早期运行中，作者观察到 evaluator 会识别出合法问题，然后说服自己这些问题“不是大问题”并批准工作。它还倾向于表面测试，而非探查边缘情况，导致更微妙的 bug 逃逸。

调优方法：阅读 evaluator 日志 → 找到其判断与作者判断分歧的案例 → 更新 QA prompt 以解决这些问题 → 重复多轮。

经过调优后，evaluator 能够发现相当具体的问题。以下是 Retro Game Maker 项目中 evaluator 发现的部分示例：

合同标准	Evaluator 发现
矩形填充工具允许拖拽填充区域	FAIL——工具仅在拖拽起止点放置 tile，而非填充整个区域。 <code>fillRectangle</code> 函数存在但在 <code>mouseUp</code> 时未正确触发。
用户可以选择并删除已放置的实体生成点	FAIL—— <code>LevelEditor.tsx:892</code> 的 <code>Delete</code> 键处理要求 <code>selection</code> 和 <code>selectedEntityId</code> 同时设置，但点击实体只设置后者。条件应为 <code>selection (selectedEntityId && activeLayer === 'entity')</code> 。
用户可以通过 API 重排动画帧	FAIL—— <code>PUT /frames/reorder</code> 路由定义在 <code>{frame_id}</code> 之后，FastAPI 将“reorder”匹配为整数 <code>frame_id</code> 并返回 422 错误。

表 2: Evaluator 发现的具体 bug 示例（来源：原文）

4.4 Retro Game Maker 对比实验

作者使用相同的 prompt 分别运行单 agent 和完整 harness：

“Create a 2D retro game maker with features including a level editor, sprite editor, entity behaviors, and a playable test mode.”

Harness 类型	时长	成本
Solo agent	20 min	\$9
Full harness	6 hr	\$200

表 3: Solo vs. Full Harness 的时长和成本对比

Harness 的成本是 solo 的 20 倍以上，但产出质量差异立竿见影：

- **Solo agent**：初看似乎符合预期，但深入使用后问题大量涌现——布局浪费空间，工作流程僵硬且缺乏引导，最关键的是**游戏本身无法运行**（实体出现在屏幕上但不响应输入，entity 定义和 game runtime 之间的接线断裂）
- **Full harness**：Planner 从一句 prompt 扩展为 16 个 feature、10 个 sprint 的完整规格（包括精灵动画系统、行为模板、音效音乐、AI 辅助的精灵生成器和关卡设计师、游戏导出等）。产出的应用**核心功能可用**——玩家可以移动角色、进行游戏，虽然物理引擎有粗糙之处（角色跳上平台后会与之重叠）

4.5 本章小结

三 agent 架构通过角色分离解决了不同层面的问题：Planner 解决规格不足，Generator 聚焦实现，Evaluator 确保质量。Sprint contract 机制弥合了高层规格与可测试实现之间的间隙。虽然成本显著增加（20x），但在核心功能的可用性上实现了质的飞跃。

5 迭代简化 Harness

第一版 harness 效果令人鼓舞，但也笨重、缓慢、昂贵。下一步的逻辑是：在不降低性能的前提下简化 harness。

Harness 简化的核心原则

“Find the simplest solution possible, and only increase complexity when needed.” ——来自 Anthropic 的 Building Effective Agents 博客文章
实际操作上，作者首先尝试了激进的简化但失败了（无法复现原有性能），随后转向更系统的方法：**每次只移除一个组件**，观察其对最终结果的影响。

5.1 Opus 4.6 带来的变化

在迭代过程中，Opus 4.6 发布，其关键改进直接对应了 harness 之前需要补偿的能力缺陷：

- 更细致的规划能力
- 更持久的 agentic task 执行
- 更可靠的大型代码库操作
- 更好的代码审查和调试（自我纠错）能力
- 显著改善的 long-context retrieval

这些改进意味着之前 harness 中的许多组件可能不再是必要的。

5.2 移除 Sprint 结构

作者首先移除了 sprint 构造。Sprint 结构的原始目的是将工作分解为模型可以连贯处理的小块。Opus 4.6 的能力提升使得模型可能无需这种分解就能原生处理任务。

关键变化：

- **Planner 保留**：没有 planner，generator 会不充分地确定范围，直接开始构建，产出功能较少的应用
- **Evaluator 移至末尾**：从每个 sprint 评分改为整体运行结束后的单次评估
- **Evaluator 的价值变为任务依赖**：在 4.5 上，构建处于 generator 能力边缘，evaluator 普遍有价值；在 4.6 上，模型能力边界外移，许多任务 generator 已能独立处理好，evaluator 仅在超出 generator 能力边界的任务部分仍有提升

Evaluator 不是固定的“有/无”决策

Evaluator 的价值取决于任务是否处于当前模型能力的边缘。模型越强，evaluator 有价值的任务范围越小，但在那些仍处于边缘的任务上，evaluator 依然提供显著提升。这意味着随着模型演进，harness 设计者需要持续重新评估每个组件的贡献。

5.3 DAW 构建实验

使用更新后的 harness，作者测试了一个更具挑战性的 prompt：

“Build a fully featured DAW in the browser using the Web Audio API.”

Agent & 阶段	时长	成本
Planner	4.7 min	\$0.46
Build (Round 1)	2 hr 7 min	\$71.08
QA (Round 1)	8.8 min	\$3.24
Build (Round 2)	1 hr 2 min	\$36.89
QA (Round 2)	6.8 min	\$3.09
Build (Round 3)	10.9 min	\$5.88
QA (Round 3)	9.6 min	\$4.06
Total	3 hr 50 min	\$124.70

表 4: DAW 构建各阶段的时长与成本 (Updated Harness + Opus 4.6)

关键观察：

- Builder 在没有 sprint 分解的情况下连贯运行超过 **2 小时**——这在 Opus 4.5 上是不可能的
- QA 仍然发现了真实缺陷，例如第一轮反馈指出：多个核心 DAW 功能仅为展示性而无交互深度（clip 不能在 timeline 上拖动，无乐器 UI 面板，无视觉效果编辑器）
- 第二轮 QA 继续发现：音频录制仍为 stub、clip 缩放和分割未实现、效果可视化仅为数字滑块而非图形化
- 最终产出是一个功能完整的浏览器 DAW：工作的 arrangement view、mixer 和 transport，用户可以完全通过 prompt 与内置 AI agent 交互来创作歌曲

Harness 版本	时长	成本	特点
Solo agent (4.5)	20 min	\$9	核心功能不可用
Full harness (4.5)	6 hr	\$200	核心功能可用，有粗糙之处
Updated harness (4.6)	3 hr 50 min	\$125	更简洁，同等或更高质量

表 5: 三个版本 harness 的综合对比

5.4 本章小结

从 Opus 4.5 到 4.6 的模型升级使得 harness 可以显著简化：sprint 结构被移除，evaluator 改为末尾单次评估。成本从 \$200/6hr 降至 \$125/4hr，同时保持甚至提升了产出质量。核心方法论是：系统地逐个移除组件并观察影响，而非激进地一次性简化。

6 总结与延伸

6.1 作者的核心洞察

Harness 设计的演化论

“The space of interesting harness combinations doesn’t shrink as models improve. Instead, it moves, and the interesting work for AI engineers is to keep finding the next novel combination.”

有趣的 harness 组合空间不会随着模型改进而缩小——它会**移动**。AI 工程师的核心工作是持续寻找下一个有价值的组合。

作者提炼的三个实践建议：

1. **实验驱动**：用目标模型在真实问题上实验，阅读 trace，调优性能
2. **按需分解**：对于更复杂的任务，将任务分解并对每个方面应用专门的 agent 可以带来额外收益
3. **模型升级时重审 harness**：剥离不再承载性能的组件，添加新组件以达成之前不可能的能力

6.2 结构化提炼

本文的贡献可以从三个层次理解：

方法论层面：建立了“假设-验证-迭代”的 harness 开发框架。每个 harness 组件都是一个可测试的假设（“模型不能独立完成 X”），当模型进步时假设需要重新验证。

架构层面：从 GAN 到 multi-agent 流水线的迁移展示了一个通用模式——将人类软件工程实践（代码审查、QA、sprint planning）编码为 agent 角色，而非试图让单个 agent 同时承担所有角色。

工程实践层面：

- 评分标准本身就是有效的 prompt 工程——即使没有 evaluator 反馈，仅将评分标准写入 prompt 就能提升 baseline
- 文件作为 agent 间通信媒介是一种简单而有效的设计
- Planner 应保持高层次以避免错误级联，sprint contract 负责弥合细节间隙
- QA agent 的校准是一个需要多轮人工审查的调优过程，而非一次性配置

6.3 拓展阅读

- Anthropic: *Building Effective Agents* —harness 设计的基础原则
- Anthropic: *Context Engineering* —上下文窗口管理策略
- Claude Agent SDK 文档—构建 multi-agent 系统的官方框架
- Playwright MCP —让 agent 与真实浏览器交互的工具